

Institución Universitaria del Caribe

Trabajo de Investigación

Diagramas UML

Docente

Neider Hernández

Alumno

Emmanuel Sánchez

Asignatura

Programación orientada a objetos

Grupo

G-1L

Fecha

04 de septiembre de 2020

¿Qué es un diagrama UML?

Un diagrama UML es la forma de visualizar sistemas y software utilizando el lenguaje unificado de modelado (UML). Se crea para comprender la arquitectura del código y la implementación propuesta de sistemas completos.

ventajas de los diagramas UML

1. Facilita la comprensión de ideas y sistemas completos

Los diagramas UML facilitan la comprensión de ideas abstractas y sistemas de software mediante visualización. Esto es beneficioso para la creación de software.

2. convierte el código completo en un diagrama visual

La creación de un programa informático suele requerir miles de líneas de código completo con relaciones y referencias en su interior lo que hace que interpretar este código sea difícil y lleva mucho tiempo. El uso de UML simplifica este proceso convirtiendo el sistema de codificación en diagramas visuales fáciles de entender.

1. Diagramas de casos de uso

un diagrama de casos de uso representa las funciones que ofrece un sistema y cómo interactúan los usuarios o sistemas externos con esas funciones.

- Conceptos: caso de uso, actor, sistema y relaciones.

- Ejemplo: En un sistema bancario: consultar saldo, retirar dinero y transferir dinero.

- Casos de uso: se utilizan para identificar los requisitos funcionales del sistema.

- Actores: cliente, administrador, empleado u otro sistema externo.

- Relaciones:

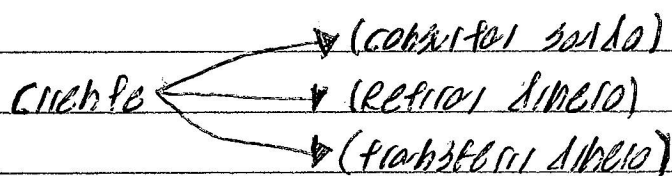
- Asociación: conecta un actor con un caso de uso.

- Incluye: un caso de uso necesita obligatoriamente otro.

- Extiende: agrega un comportamiento adicional.

- Generalización: un actor o caso de uso hereda características de otro.

- Diagrama UML: (sistema bancario)



2. Diagrama de clases

Un diagrama de clases representa un sistema estático orientado a objetos. Define objetos por clases, atributos y funciones. Es un bloque de construcción fundamental de cualquier solución orientada a objetos. Muestra las clases de un sistema y sus dependencias.

• Conceptos: clase, atributo, método, objeto y visibilidad.

• Ejemplo: una clase Usuario con los atributos nombre, correo y el método `iniciarSesion()`.

• casos de uso: Diseño de software, POO, y definición de la arquitectura.

• Actores: No utiliza actores como elemento principal. Representa las clases que forman el sistema.

• Relaciones:

- Asociación: una clase se relaciona con otra.
- Herencia: una clase hereda de otra.
- Agregación: una clase contiene otras.
- Composición: una clase contiene otras y estas dependen de ella.
- Dependencia: una clase utiliza temporalmente otra.

• Diagrama UML:

Usuario
- nombre
- correo
+ iniciarSesion()

3. Diagrama de secuencia

Un diagrama de secuencia muestra cómo los distintos objetos se relacionan e interactúan entre sí en un sistema.

- **Conceptos:** Objetos, líneas de vida, mensajes y activaciones.

- **Ejemplo:** para iniciar sesión: usuario -> sistema -> bd -> sistema -> usuario.

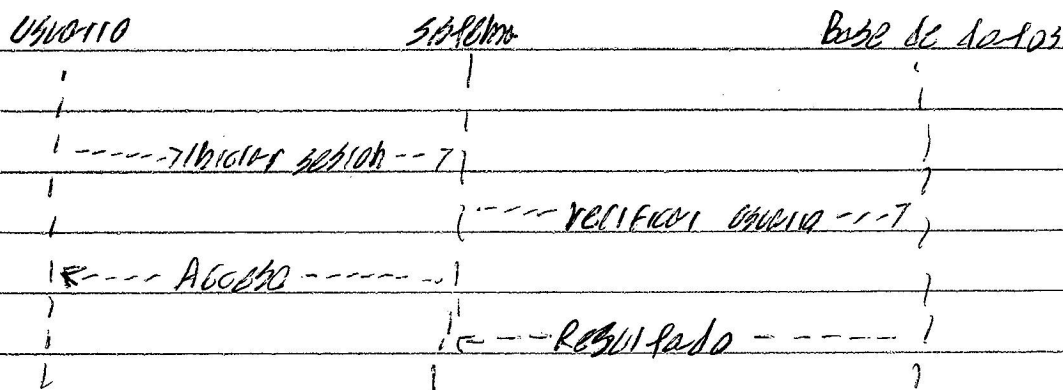
- **Casos de uso:** describir paso a paso el funcionamiento de un sistema.

- **Actores:** usuarios, objetos del sistema y sistemas externos.

- **Relaciones:** se representan principalmente mediante mensajes entre los participantes.

- **Diagrama UML:**

Iniciar sesión



II. Diagrama de actividades

Un diagrama de actividades divide los procesos grandes en actividades más pequeñas. Se utiliza para analizar en cualquier componente de un sistema.

• Conceptos: Actividades, acción, decisión, Inicio, Fin y Flujo.

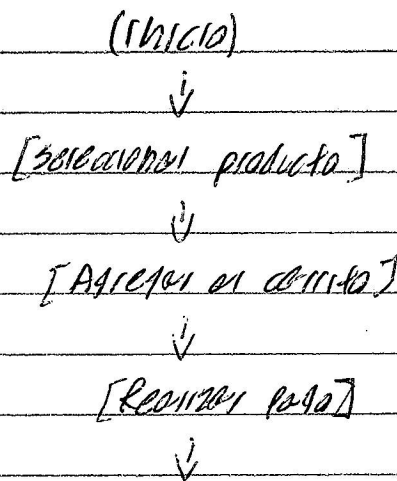
• Ejemplo: comprar un producto: seleccionar producto -> agregar al carrito -> pagar -> ¿pago aprobado? -> confirmar pedido.

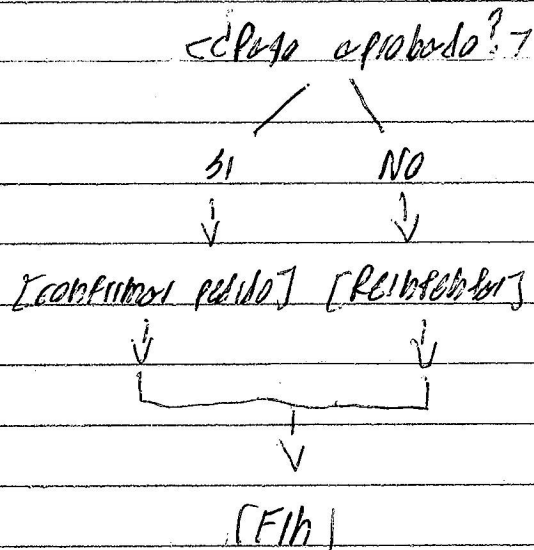
• Casos de uso: procesos empresariales asignados a Flujos de trabajo.

• Actores: pueden aparecer usuarios o departamentos responsables de determinadas actividades.

• Relaciones: se utilizan Flujos de control, decisiones y bifurcaciones.

• Diagrama UML





5. Diagrama de estados

Un diagrama de estado muestra los comportamientos de los distintos componentes de un sistema. Se puede visualizar cómo los elementos actúan de forma diferente según el estado del programa que se está desarrollando.

• Conceptos: Estado, transición, evento, condición + acción.

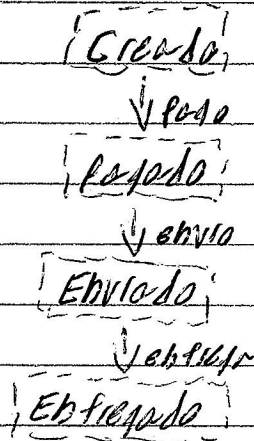
• Ejemplo: Un pedido puede estar en creado -> pagado -> enviado -> entregado.

• casos de uso: sistemas donde los objetos cambian de estado dependiendo de eventos.

• Acciones pueden ser acciones o sistemas que generan eventos.

• Relaciones: se representan mediante transiciones entre estados.

° Diagrama UML



6. Diagrama de componentes

un diagrama de componentes ofrece a los desarrolladores una comprensión global de los objetos físicos de un sistema. Este tipo muestra la relación estructural entre cada componente físico y subcomponente de un sistema de software completo.

° conceptos: componente, interfaz, dependencia y cohesión.

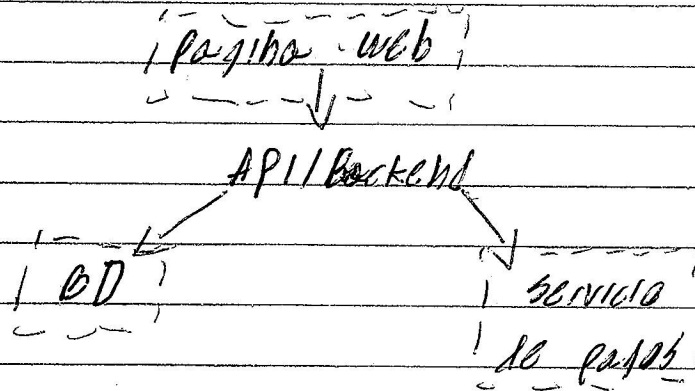
° Ejemplo: un sistema puede tener componentes como interfaz web, API, servicio de papeo y base de datos.

° casos de uso: Arquitectura y organización de aplicaciones globales.

° Actores: Los usuarios y sistemas externos pueden interactuar con los componentes.

° Relaciones: Dependencias, interfaces y cohesiones entre componentes.

° Diagrama UML:



1. Diagrama de Despliegue

Los diagramas de despliegue muestran la relación entre los componentes de software y hardware de un sistema. Representa la disposición física de los nodos en un sistema distribuido. Estos diagramas son útiles para software que funcione en varios sistemas de hardware diferente.

° Conceptos: Nodo, dispositivo, servidor, artefacto y conexión.

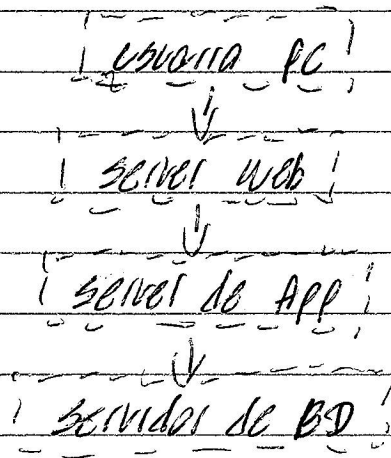
° Ejemplo: una aplicación web desplegada en un servidor web; un servidor de aplicaciones y un servidor de base de datos.

° casos de uso: diseño de infraestructura y arquitectura física.

° Actores: dispositivos, servidores y sistemas externos.

° Recursos: conexiones y dependencias entre nodos.

° Diagrama UML



B. Diagrama de objetos

Los diagramas de objetos son similares a los de clases, la diferencia es que estos utilizan ejemplos del mundo real. Los diagramas de objetos también se denominan diagramas de instancia porque muestran el estado del sistema en un momento determinado.

° Conceptos: objeto, atributo y enlace.

° Ejemplo: si la clase Usuario existe, entonces un objeto podría ser Usuario:Usuario, con nombre Carlos.

° casos de uso: Muestra ejemplos concretos de la estructura de un sistema.

° Actores: No utiliza actores como elemento principal

° Relaciones: Enlaces entre objetos que representan relaciones entre clases.

° Diagrama UML:

```
classDiagram
    class Usuario {
        nombre "correo"
        correo = "correo@mail"
    }
```

4. Diagrama de paquetes

Los diagramas de paquetes son diagramas que agrupan las clases en paquetes mostrando las distintas dependencias y relaciones entre los paquetes de un sistema.

° Conceptos: paquete, elemento y dependencia.

° Ejemplo: un sistema puede dividirse en paquetes, clases, vistas, reportes y repositorios.

° Accesos no son un elemento principal.

° Relaciones: principalmente dependencia entre paquetes.

° Diagrama UML

```
graph LR
    Usuarios[Usuarios] --- Vistas[Vistas]
    Vistas --- Paquetes[Paquetes]
    Paquetes --- Reportes[Reportes]
    Reportes --- Vistas
```

1a. Programa de comunicación

Los diagramas de comunicación se centran en los mensajes que se transmiten entre diferentes objetos. Mientras que crean un mapa completo y general del producto.

- conceptos: objetos, enlaces y mensajes.

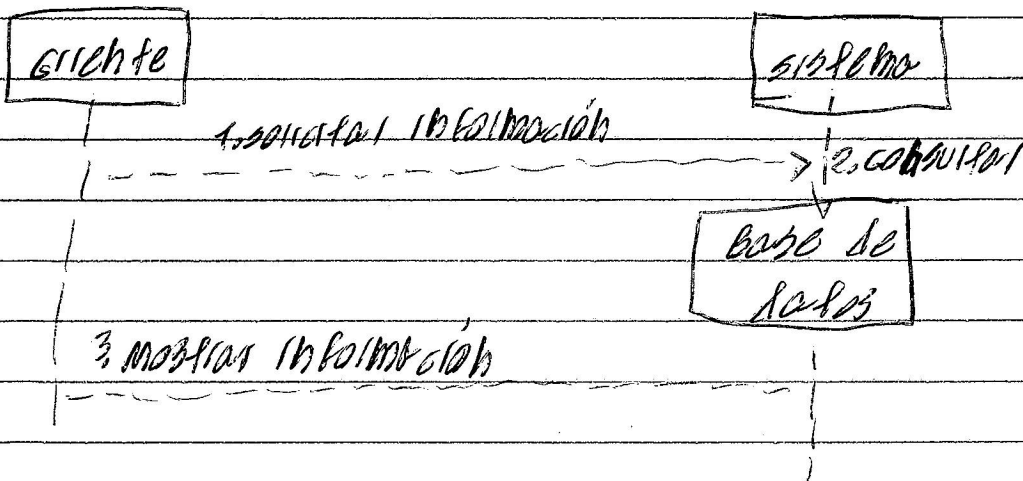
- Ejemplo: un cliente envía una solicitud a sistema de computo bajo la datos.

- casos de uso: muestran cómo utilizan diferentes objetos para completar una aplicación.

- Actores: pueden participar usuarios y objetos del sistema.

- Relaciones: Enlaces y mensajes numerados entre objetos.

- Diagrama UML



11. Diagrama de estructura compuesta

Representa la estructura interna de una clase, componente o elemento, mostrando sus partes y cómo interactúan entre sí.

• concepto: clase, partes, interfaces, interfaces y relaciones.

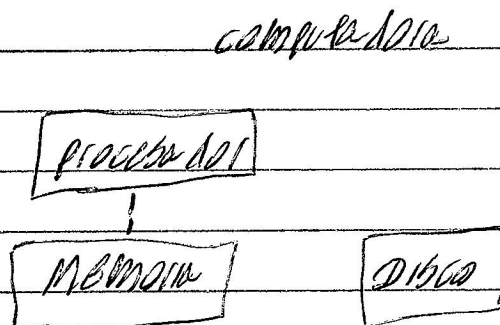
• Ejemplo: un sistema de computadora puede tener una clase computadora formada internamente por procesador, memoria y disco.

• casos de uso: mostrar como esta formada internamente un componente o la interacción entre las partes de una clase.

• Acciones: No utiliza acciones como elemento principal.

• Relaciones: conexiones entre partes, interfaces, dependencias y comunicación entre componentes internos.

• Diagrama UML



12. Diagrama de temporización

Representa como cambia el estado o comportamiento de uno o varios objetos a lo largo del tiempo. Es especialmente útil cuando el tiempo es un elemento importante del sistema.

• Conceptos: tiempo, estados, eventos, objetos.

• Ejemplo: Una puerta automática puede estar en los estados cerrada-Abriendo-Abierta-Cerrando-Cerrada.

• casos de uso: sistemas donde el tiempo es importante, control de dispositivos o sistemas electrónicos.

• Actores: puede representar usuarios, dispositivos u objetos que provocan eventos.

• Relaciones: Eventos que provocan cambios, transiciones entre estados, cambios de estados a través del tiempo.

• Diagrama UML

tiempo - - - - - 7

Puerta:

